

Research Progress: Hybrid Seccomp Profiling

Jiwoo Ahn, Byungchul Tak

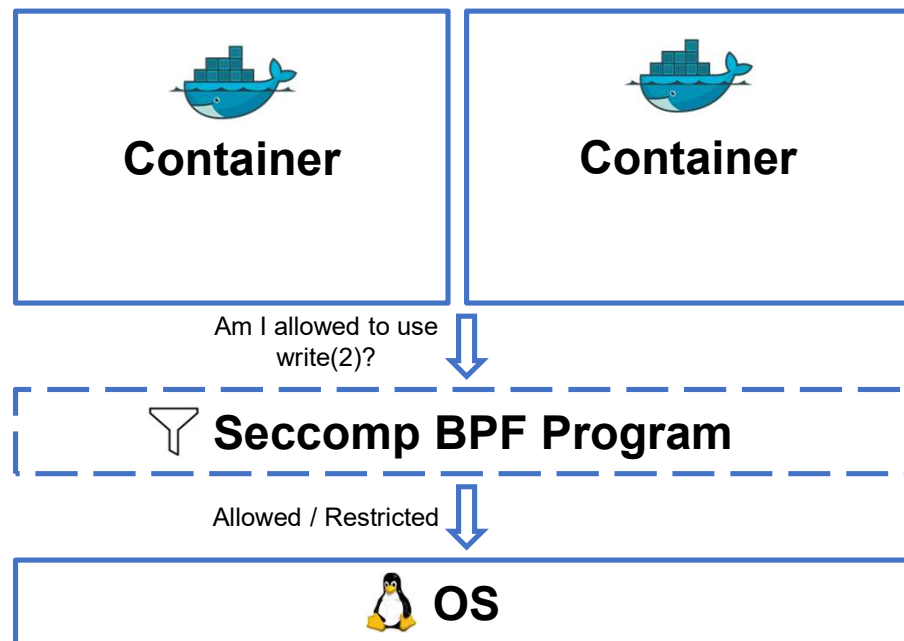
CLODIS Lab, KNU

July 13th, 2026



What is Seccomp?

- **Seccomp: Linux security mechanism that restricts the system calls a process is allowed to use**
 - By allowing only necessary syscalls and blocking the others, seccomp can reduce the attack surface of a system.



Why is Seccomp Profiling Challenging?

- **Identifying the exact set of syscalls required by a program is challenging.**
- Existing approaches are usually divided into two categories:

Static analysis

- Analyze the source code or binary
- It can reason about more execution path
 - ***Considered complete***
- Some analyze path maybe unreachable
 - Over-approximated and lower in precision

Dynamic analysis

- Observe syscalls that are actually invoked during execution
 - ***Considered precise***
- Cannot guarantee coverage of all possible path:
 - Suffers from incompleteness

Our Initial Approach

- Our original goal: **Enhancing the dynamic analysis using LLMs.**
- **Utilize LLMs to generate test cases that can trigger as many execution paths as possible**
- We aimed to compensate for the main weakness of dynamic analysis:
Low completeness.

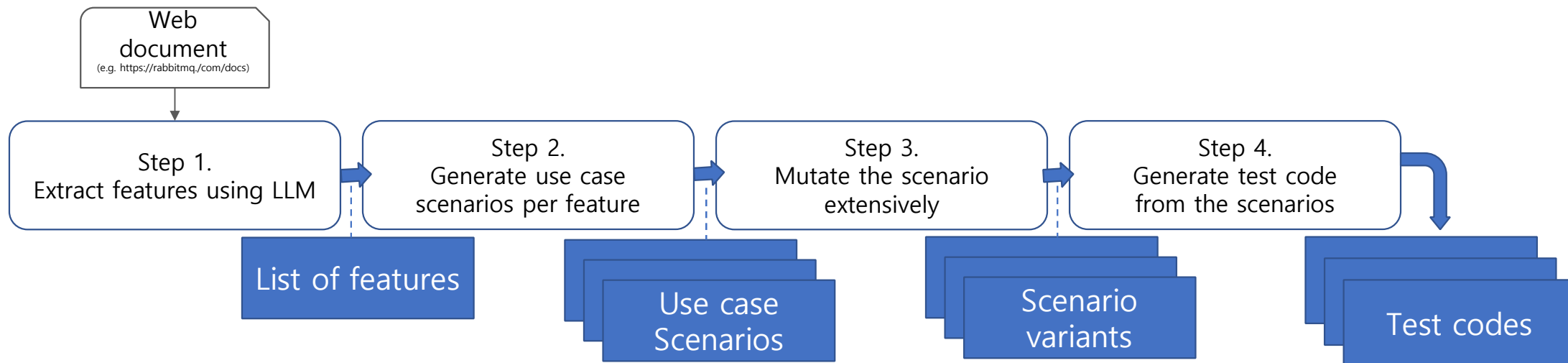


Fig. Initial Architecture - Enhancing Dynamic Analyzer

Primary Testcode Performance

- **Conducted initial experiments on 5 real world systems**
 - We generated test cases and extracted the syscalls observed during execution:
- **The results show mixed behaviors:**
 - Each tool detected some syscalls that the others missed.
- **Number of allowed system calls: prior studies vs ours**
 - *Chestnut* and *Confine* both employ static analysis

Container images	# of system calls				
	Chestnut (Sourcealyzer)		Confine		Ours
	found	intersection	found	intersection	
Redis	87	56	148	68	70
Nginx	123	69	170	75	76
Memcached	98	42	150	46	48
Apache Httpd	94	55	151	63	65
Postgresql	113	78	249	91	93

Naturally leads to this question: What is the correct syscall set? → **Ground Truth**

Ground Truth Generation

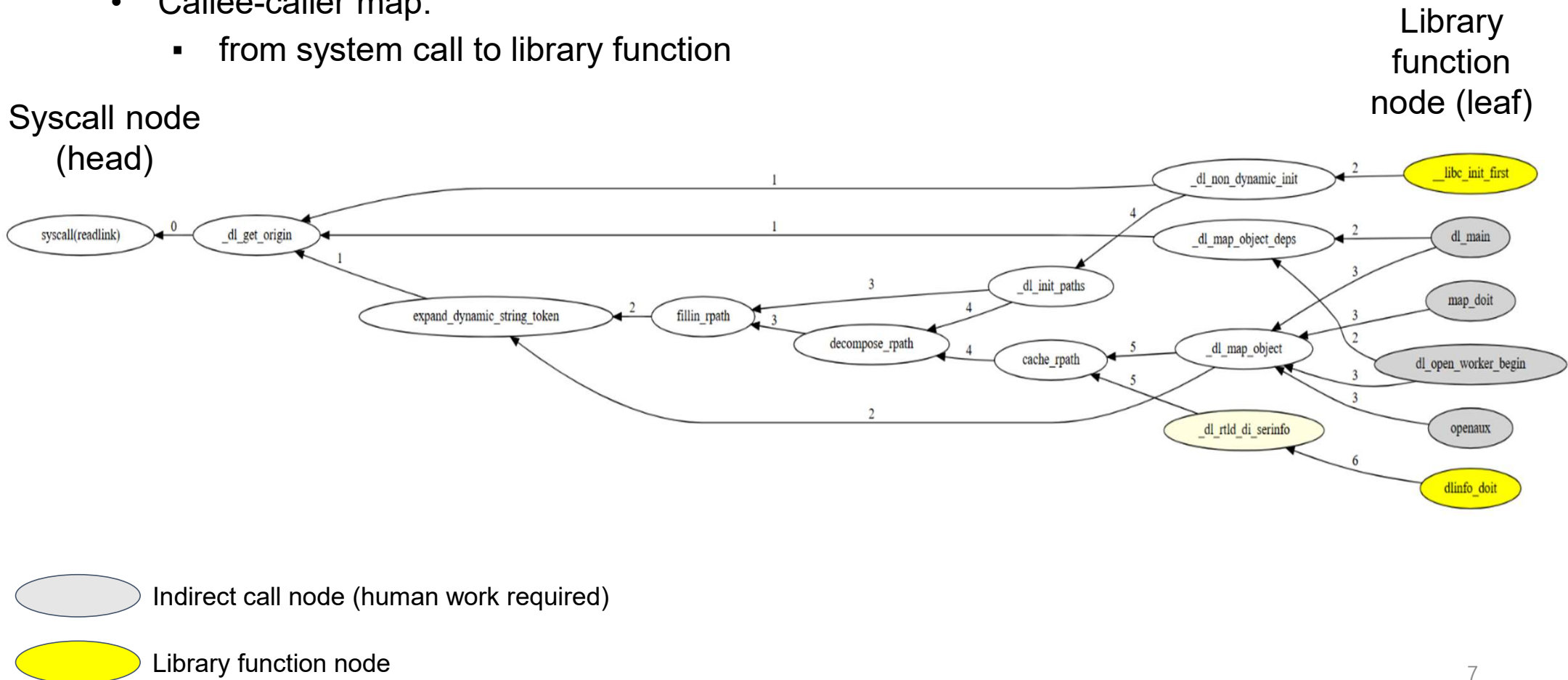
- **Our strategy for GT generation:**
 - Glibc is the most popular library → We focus on Glibc
 - We build libc-function-to-syscall mapping
- **Initial Approach:**
 - Building callee-to-caller chain from each syscall in glibc
 - And manually resolve indirect calls
 - ▶ Expectation: small manual effort

Indirect calls (icall)?:
Call sites where targets
determined at runtime

ex) function pointers,
jump tables, dynamic
loader etc.

Example: Ideal (readlink syscall, 22 edges)

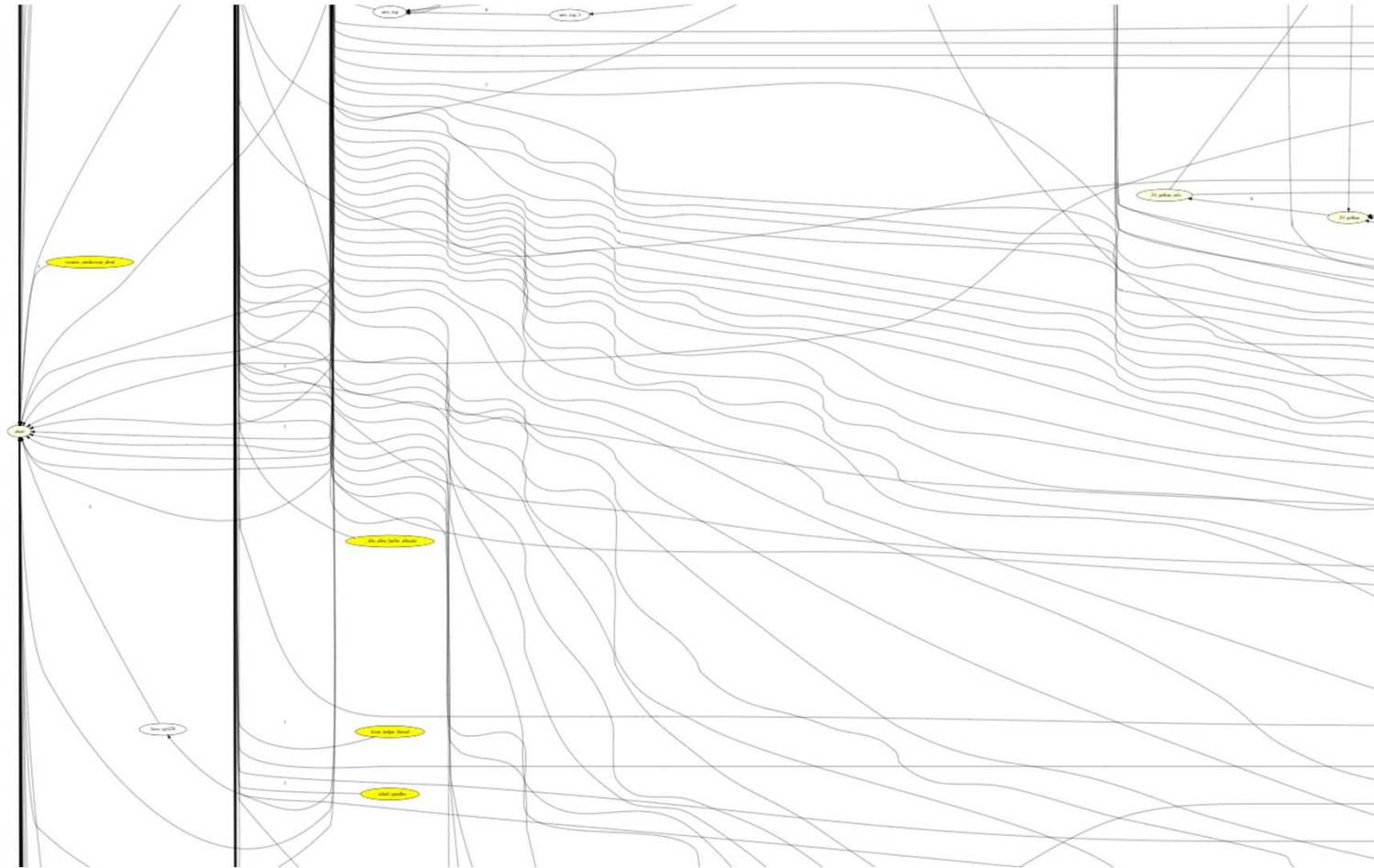
- Callee-caller map:
 - from system call to library function



Challenge in Ground Truth Generation

- **Manual effort too heavy!!**
 - 1,607 reachable icalls in glibc 2.41
- **Some indirect calls can not be resolved statically**
 - User defined callback functions
 - Functions from external libraries
 - Dynamically loaded modules
 - ▶ e.g., NSS/ gconv/ locale etc.
 - Runtime selected implementations
 - ▶ e.g., OS/ arch specific variants

Example: Reality (gettid syscall, 6595 edges)



Resolving Indirect calls with LLM

- Exploring whether LLMs can help resolve indirect calls.
- Caveats on using LLM:
 - LLM may hallucinate
 - LLM outputs can vary across runs
 - ▶ Cannot trust a single response as-is
- Our Approach:
 - **Building consensus across different LLMs**
 - Resolution criterion: multiple models give the same answer.

Initial Results

- **Icall resolution results by each model**

- *Resolved*: Concrete callee target identified
- *External*: Statically unresolved external target e.g.) user callback, dynamic library
- *Not icall*: Not an icall site

	GPT	Claude	Gemini
Resolved	796 (49.5 %)	542 (33.7 %)	546 (34.0 %)
External	789 (49.0 %)	1,046 (65.0 %)	1,048 (65.2 %)
Not icall	22 (1.3 %)	19 (1.18 %)	13 (0.8 %)

- **Initial inter-model comparison:**

616 + 62 = 678 disagreement cases

Pattern	Count (Rate)
Full consensus	929 / 1,607 (57.2%)
2:1 Agreement	616 / 1,607 (39.0 %)
All different	62 / 1,607 (3.9 %)

Disagreement Resolution

- **678** cases with different responses across models

Our Strategy: Re-query models to retain or revise their answer **toward Full Consensus**

- **Methodology:**
 - We ask whether to retain its answer
 - Input prompt:
 - ▶ Related source snippet
 - ▶ Its original answer and reasons
 - ▶ Other model's answer and reasons
 - Repeat until agreement is reached

Consensus rate across refinement rounds

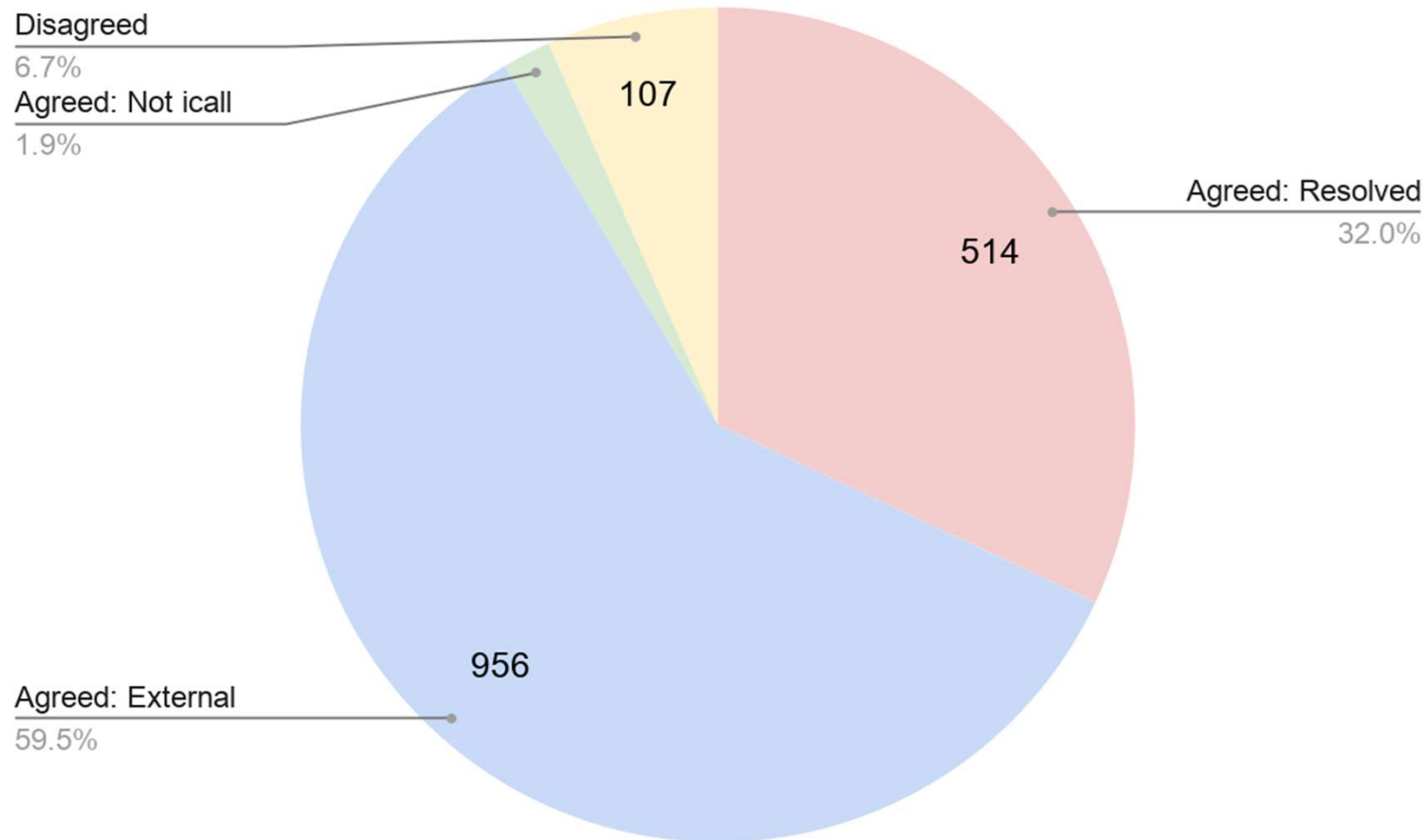
- Consensus rate improved across rounds
- Filter removes syntactically incorrect LLM answer

Rounds	Consensus count	Consensus rate	Increment
Round0	929 / 1,607	57.8 %	-
Round0 + filter	992 / 1,607	61.7 %	+ 63
Round1	1,315 / 1,607	81.8 %	+323
Round1 + filter	1,315 / 1,607	81.8 %	+ 0
Round2	1,456 / 1,607	90.6 %	+ 141
Round2 + filter	1,456 / 1,607	90.6 %	+ 0
Round3 (final)	1,500 / 1,607	93.3 %	+ 44

- **107 disagreements remains** after three rounds
 - $1,607 - 1,500 = 107$

Current Breakdown After Round 3

- 1,496 / 1,607 cases reached agreement after refinement



Remaining Questions

No Full Consensus Cases:

Can we reduce disagreements further?

Full Consensus Cases:

What if all three models are wrong?